

Chapter 9

Scheduling

Whenever more threads are ready than there are CPUs, the scheduler decides who runs — at every timer interrupt, and whenever a thread blocks, wakes up, yields or exits. This chapter covers the metrics, the classic policies you will be asked to simulate by hand in the exam, how real systems combine them, and the question that matters most for your `pthread_join`: should a waiting thread spin, yield, or sleep?

9.1 Bursts and metrics

Threads alternate between *CPU bursts* (computing) and *I/O bursts* (waiting for the disk, the network, the keyboard, a lock). CPU-bound threads have long CPU bursts, I/O-bound (interactive) threads short ones. For a job that arrives at time a , first runs at s and finishes at f with total CPU time b :

Metric	Definition	Who cares
turnaround time	$f - a$	batch jobs
waiting time	$f - a - b$ (time spent ready, not running)	fairness
response time	$s - a$	interactive users
throughput	jobs finished per time unit	the whole system

The goals conflict: short quanta improve response and cost throughput (context switches); favouring short jobs improves the average and can starve long ones.

9.2 The classic policies

Example (arrival, burst, priority): A (0, 5, 2), B (1, 3, 1), C (2, 1, 3), D (4, 2, 1) — module 09, question 1. One box per time unit.

FCFS (first come, first served): run jobs to completion in arrival order. Simple, no starvation, but short jobs wait behind long ones — the *convoy effect*.

A	A	A	A	A	B	B	B	C	D	D
0	1	2	3	4	5	6	7	8	9	10

average waiting 3.75

SJF (shortest job first, non-preemptive): when the CPU becomes free, pick the job with the shortest burst. When all jobs are present from the start, this is provably optimal for the average waiting time (exchange argument: a longer job directly in front of a shorter one — swap them, and the sum of waiting times drops). With staggered arrivals it is only a good heuristic: a short job arriving just after a long one has started still has to wait.

A	A	A	A	A	C	D	D	B	B	B
0	1	2	3	4	5	6	7	8	9	10

average waiting 3.00

SRTF (shortest remaining time first): SJF with preemption — a new job with less remaining work than the running one takes over. Optimal for the average waiting time over all schedules.

A	B	C	B	B	D	D	A	A	A	A
0	1	2	3	4	5	6	7	8	9	10

average waiting 2.00, response 0.25

Both need the length of the next CPU burst, which the OS does not know. It can *predict* it by exponential averaging of the past bursts: $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.

Round robin (RR): a FIFO ready queue; each job runs for at most one *quantum* q , then goes to the end of the queue. Good response time, fair, no starvation. The quantum is the knob: too small and the CPU is busy switching (a 0.1 ms switch with $q = 1$ ms wastes 9%), too large and RR degenerates into FCFS. Typical values: a few milliseconds. With $q = 2$:

A	A	B	B	C	A	A	D	D	B	A
0	1	2	3	4	5	6	7	8	9	10

average waiting 4.25, response 1.50

A tie that exams love: at $t = 2$, C arrives *and* A's quantum ends. The convention used in **module 09** (and by most textbooks): the newcomer is queued first, the preempted job behind it. State your convention when you answer.

Priority scheduling: always run the most important ready job (here: smaller number = more important; preemptive):

A	B	B	B	D	D	A	A	A	A	C
0	1	2	3	4	5	6	7	8	9	10

average waiting 3.25

Low-priority jobs can *starve* if higher-priority work keeps arriving. Remedy: *aging* — raise the priority of waiting jobs over time. Also watch for priority inversion (chapter 6).

9.3 Multi-level feedback queue

MLFQ learns what kind of job it deals with, without being told:

1. Several queues with decreasing priority; round robin within a queue; higher queues always win, and preempt lower ones.
2. A new job enters the top queue.

3. A job that uses up its time allotment on a level is moved one level down. The allotment is counted *across* I/O — otherwise a job could stay on top by blocking just before its quantum ends.
4. Periodically, all jobs are boosted back to the top: no starvation, and a job that changed its behaviour gets a new chance.

Interactive jobs (short bursts) stay high and get quick responses; CPU-bound jobs sink and share the rest. `module 09`'s bonus implements it with exact rules.

9.4 Proportional share and Linux

Instead of priorities, give each job a *share*: lottery scheduling draws tickets, stride scheduling advances a “pass” value by $1/\text{tickets}$. Linux's scheduler (CFS, since 6.6 EEVDF) is of this kind: each task accumulates *virtual runtime*, weighted by its nice value (nice 0 = weight 1024, each nice step $\approx 1.25\times$ less), and the task with the least virtual runtime runs next. A task that sleeps a lot has little virtual runtime and runs soon after waking — interactive tasks stay responsive without explicit classification (`module 09`'s examples measure both effects).

9.5 Spin, yield or sleep?

A thread that must wait for another thread (a flag, a lock, a join) on a *single* CPU:

spin (`while (!flag) {}`): burns its whole quantum every round — and the flag cannot change meanwhile, because the thread that would set it is not running. Pure waste.

yield (`while (!flag) yield();`): gives the CPU away at once, but stays *runnable*: it is scheduled again and again just to check and yield. Costs a context switch per round, forever.

sleep (block until woken): leaves the ready queue entirely; costs exactly one switch away and one back.

A kernel waits by sleeping, except for very short waits on locks held for a few instructions.

In SWEB

`Scheduler::schedule()` (`common/source/kernel/Scheduler.cpp`) is round robin: it picks the first thread in `threads_` that is `Running` (i.e. schedulable) and rotates the list. The timer interrupt (`irqHandler_0`) calls it every tick, so the quantum is one tick; `Scheduler::yield()` triggers the same code voluntarily with `int $65`. When nothing is runnable, the idle thread runs. `sleep()` sets the state to `Sleeping`, which removes a thread from consideration until `wake()` sets it back. For `pthread_join`: the joiner must *sleep* until the target has exited — a `while (!done) yield();` loop would “work” and waste every round-robin cycle.

Key points

- Metrics: turnaround $f - a$, waiting $f - a - b$, response $s - a$.
- FCFS (convoys), SJF (optimal when all jobs start together), SRTF (optimal), RR (response; quantum trade-off), priority (starvation $\rightarrow$ aging).
- MLFQ: demote on used allotment (across I/O), boost periodically.
- Linux: weighted virtual runtime, nice changes the weight.
- Wait by sleeping; on one CPU, spinning is pointless and yielding in a loop is wasteful.